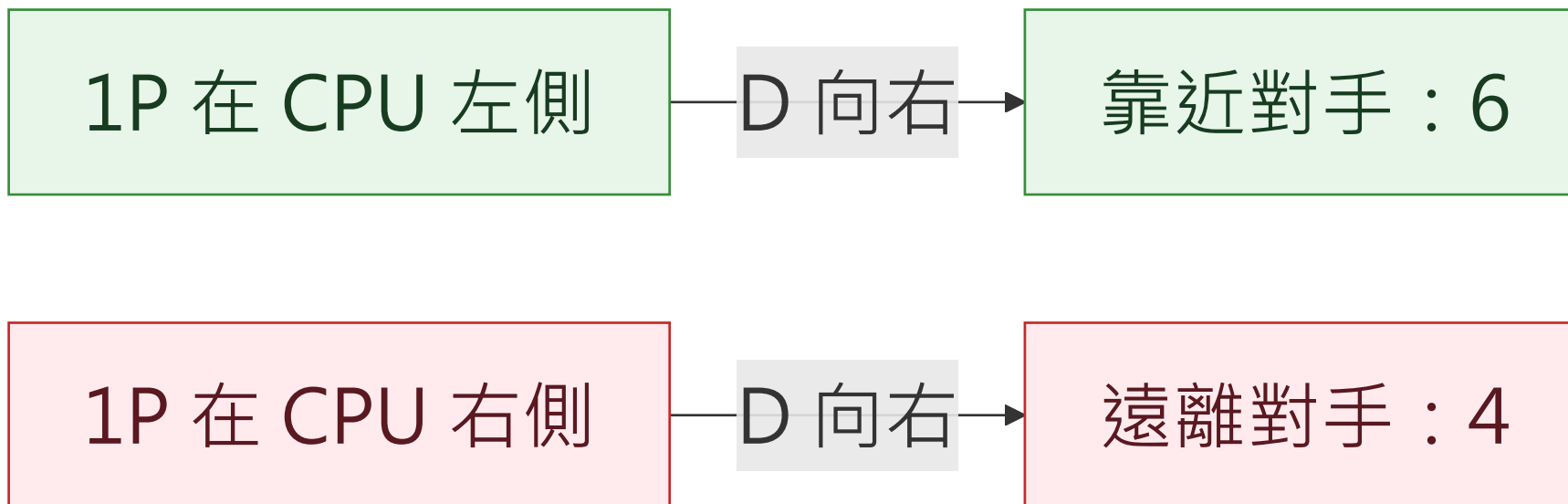


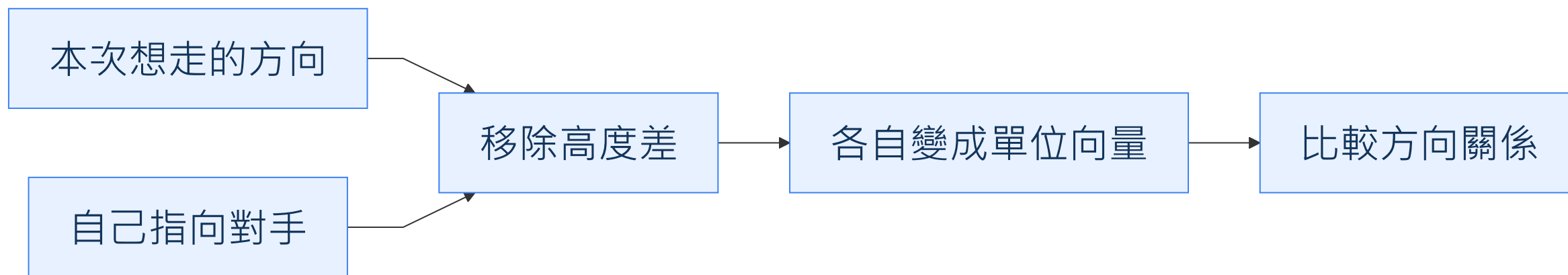
同一個 D，為什麼會從 6 變成 4？



A / D 看螢幕左右，4 / 6 看移動輸入和對手的關係。

程式定位 FightingPlayerCharacter.cpp：DoMove() / DrawCombatDebug()
 FightingCharacter.h：ERelativeDirection

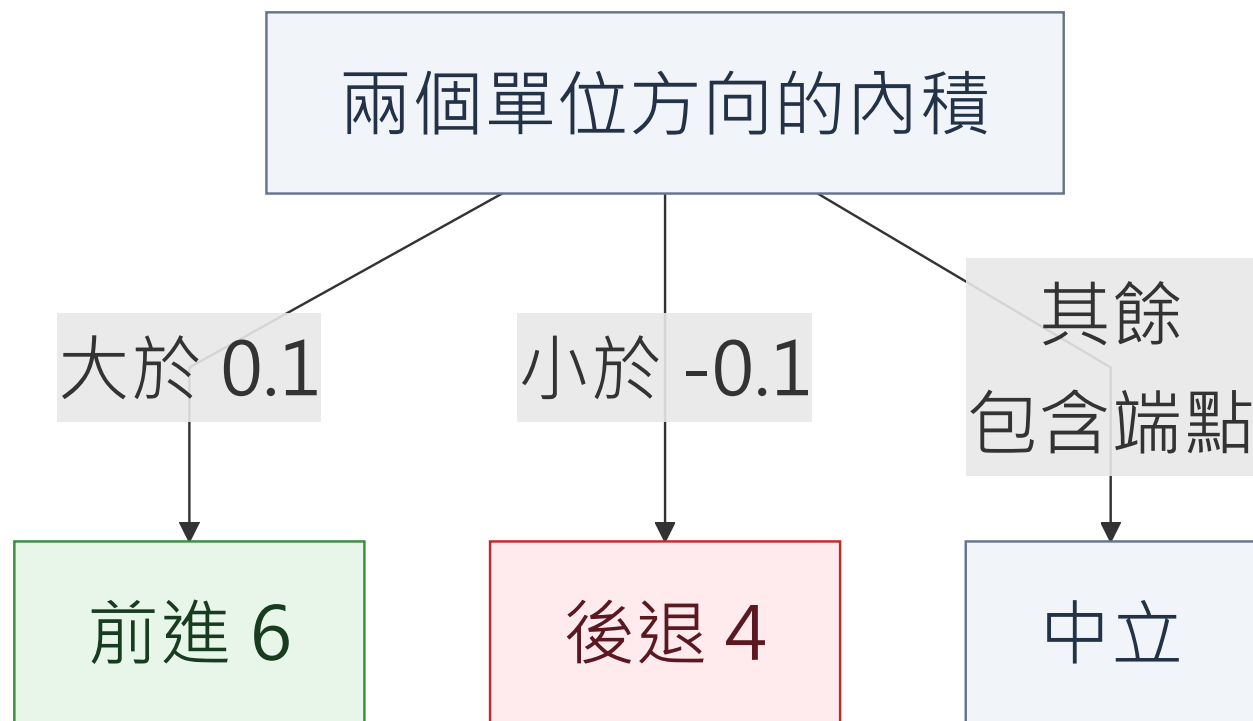
比較方向前，先把兩支箭頭整理好



先只留下水平方向，再把長度統一為 1，才不會把距離當成方向差異。

程式定位 FightingPlayerCharacter.cpp : DoMove() / MovementIntent
 FightingCharacter.cpp : ClassifyRelativeDirection() / GetSafeNormal()

內積如何分出 6、4 與中立？



內積的幾何意義

A：對手方向；B：移動輸入方向。

$$\mathbf{A} \cdot \mathbf{B} = |\mathbf{A}||\mathbf{B}|\cos \theta$$

θ 是兩個向量的夾角。

$|\mathbf{B}|\cos \theta$ ：B 沿 A 方向的有號投影。

兩向量已正規化，內積 = $\cos \theta$ 。

大於 0： $\theta < 90^\circ$

移動輸入有朝向對手的分量。

小於 0： $\theta > 90^\circ$

移動輸入有遠離對手的分量。

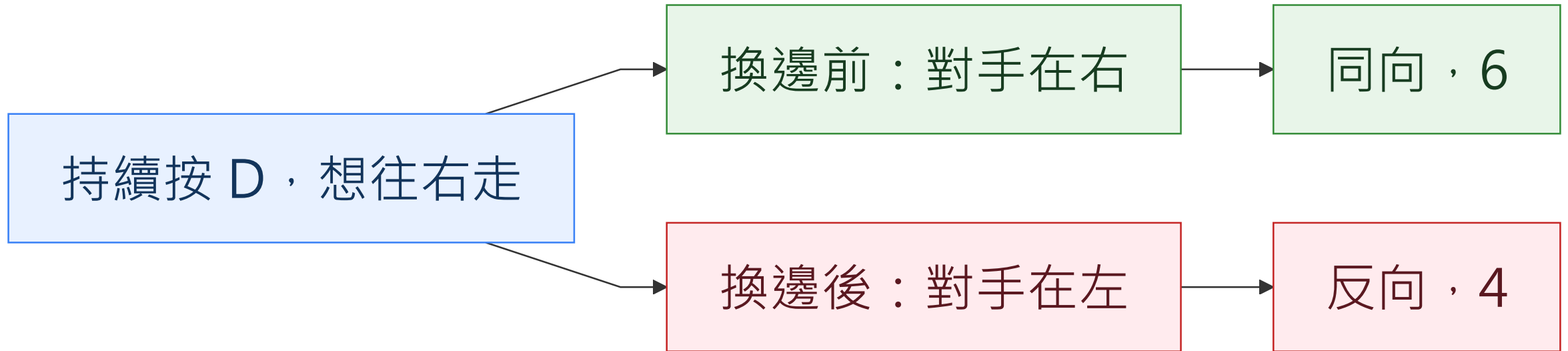
等於 0： $\theta = 90^\circ$

互相垂直，沒有朝向或背離對手的分量。

同向接近 1、反向接近 -1、垂直接近 0；教材用 ± 0.1 留出中立區間。

程式定位 `FightingCharacter.cpp`：`ClassifyRelativeDirection()` / `FVector::DotProduct()`
 `FightingCharacter.cpp`：`DirectionDot` / `RelativeDirectionDotThreshold = 0.1`

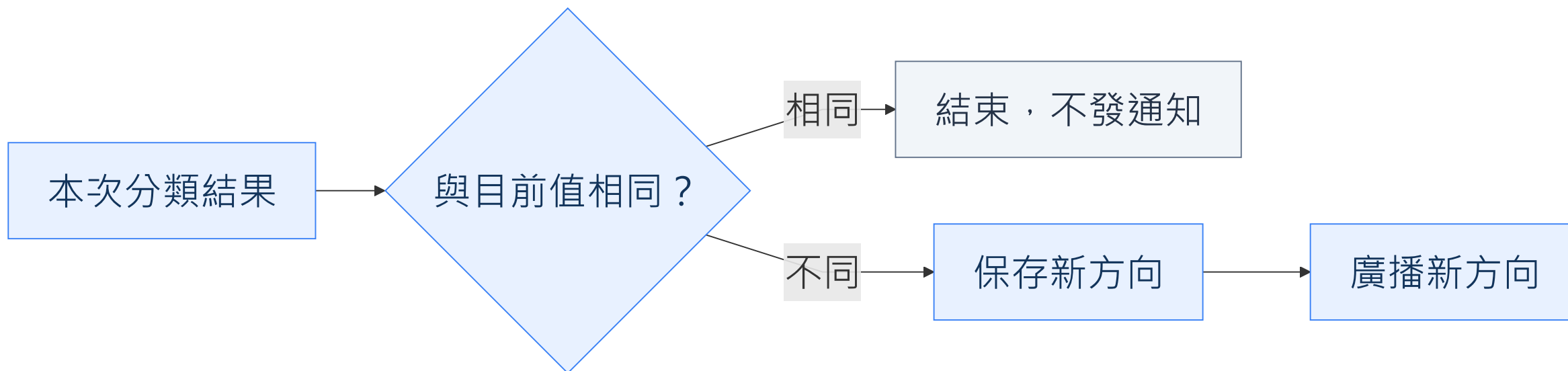
按住 D 穿越，方向資料怎麼變？



輸入持續更新；換邊使對手方向反轉，同一個按鍵就能得到新的 4 / 6。

程式定位 FightingPlayerCharacter.cpp : SetupPlayerInputComponent() / ETriggerEvent::Triggered
 FightingPlayerCharacter.cpp : Move() / DoMove()

保存目前方向，只在改變時通知



按住時持續判定，值有改變才保存並廣播。

程式定位 FightingPlayerCharacter.cpp : SetCurrentRelativeDirection() / StopMove()
 FightingPlayerCharacter.h : CurrentRelativeDirection / OnRelativeDirectionChanged

格鬥遊戲換邊 (Side Switch)



拋技換邊 | 00:09

TEKKEN 8：利用拋技交換雙方位置。

▶ 觀看影片

Cross-up Combo

TEKKEN 8：越過對手，接續另一側的攻勢。

▶ 觀看影片

Cross-under Combo

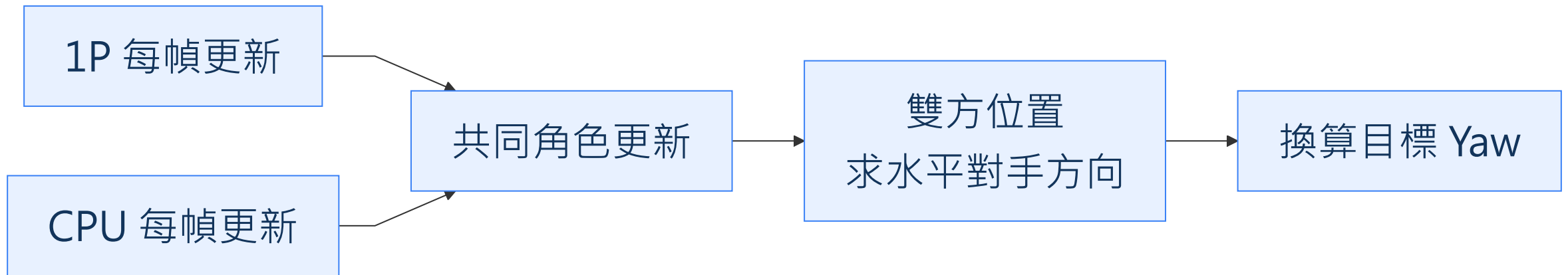
SFV：從空中的對手下方穿到另一側。

▶ 觀看影片

換邊後，前進 / 後退都要依新的對手方向判定。

程式定位 `FightingCharacter.cpp`：UpdateHorizontalOpponentDirection() / ClassifyRelativeDirection()
 `FightingPlayerCharacter.cpp`：DoMove()

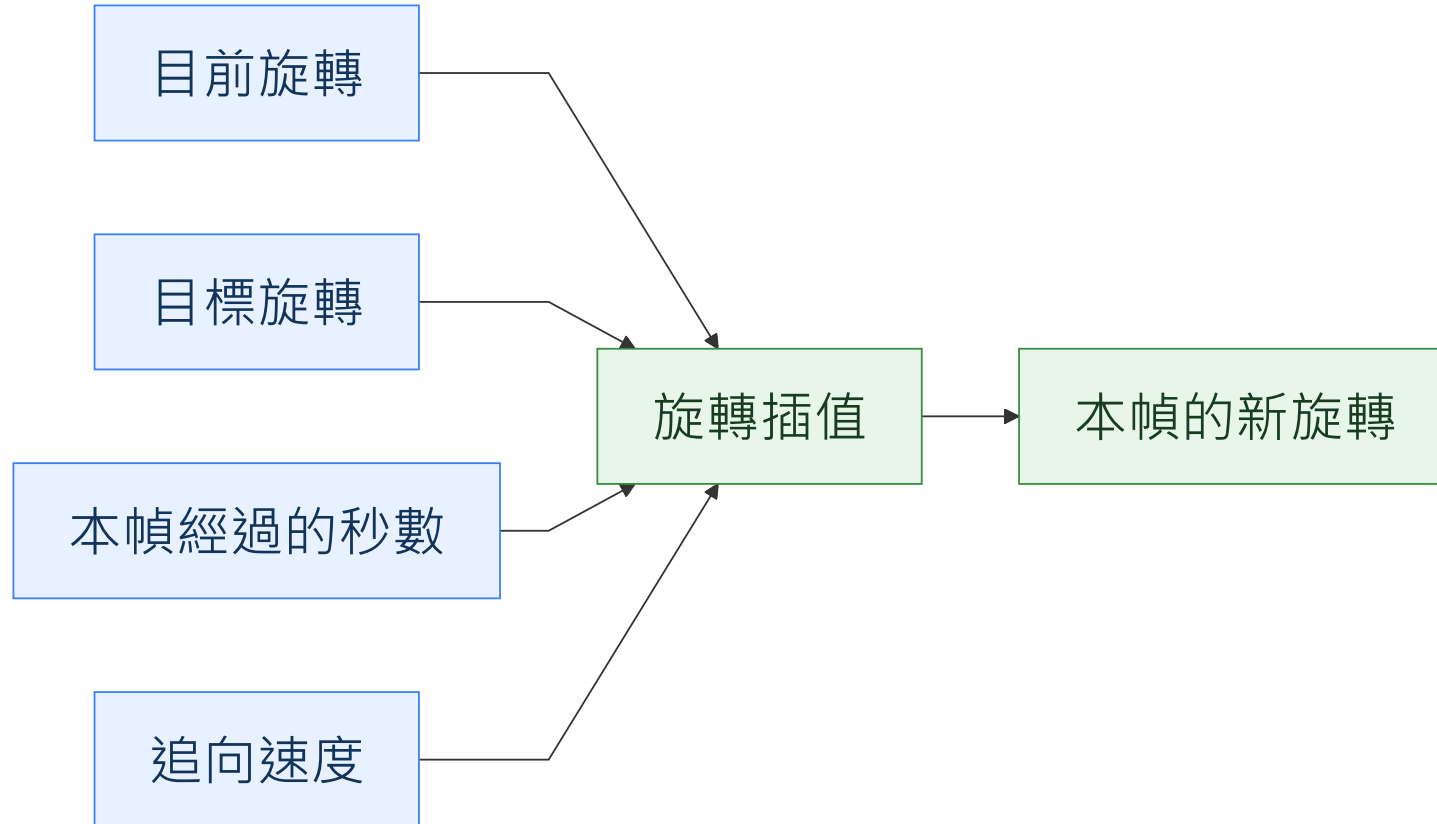
先決定想朝哪裡，再處理怎麼轉



位置先決定目標角度；算出目標後，才處理角色目前的旋轉。

程式定位 FightingCharacter.cpp : Tick() / RefreshOpponentDirection()
 FightingCharacter.h : LastValidOpponentDirection / DesiredFacingYaw

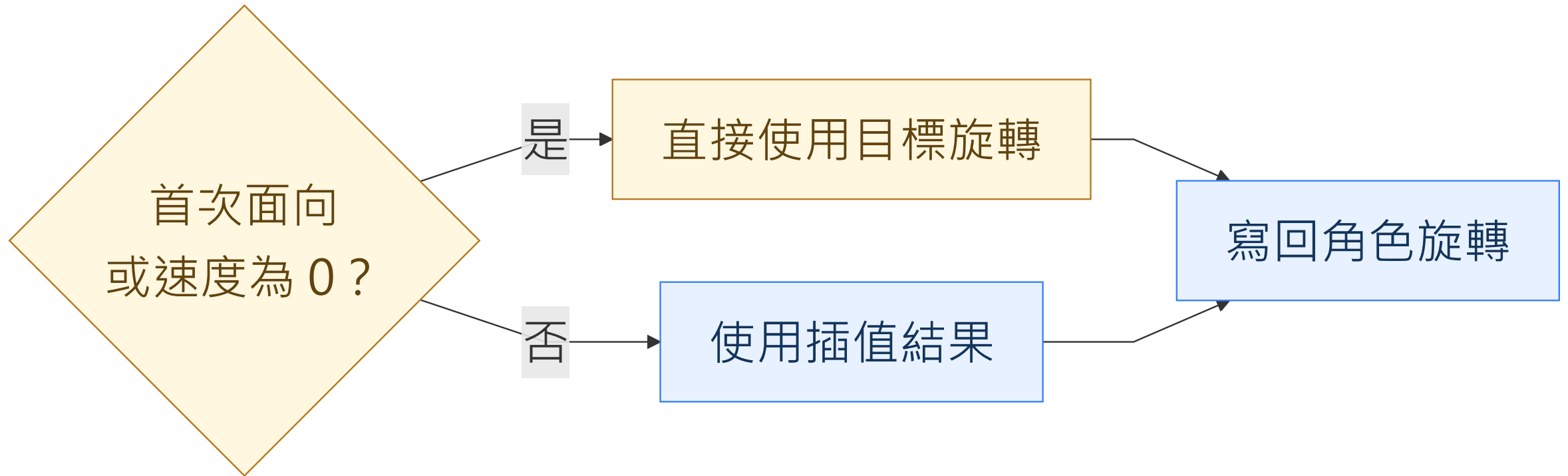
用插值把轉身過程攤開來看



目標告訴角色要朝哪裡，插值決定這一幀先轉到哪裡。

程式定位 `FightingCharacter.cpp` : `UpdateFacing()` / `FMath::RInterpTo()` / `SetActorRotation()`
 `FightingCharacter.h` : `FacingInterpolationSpeed`

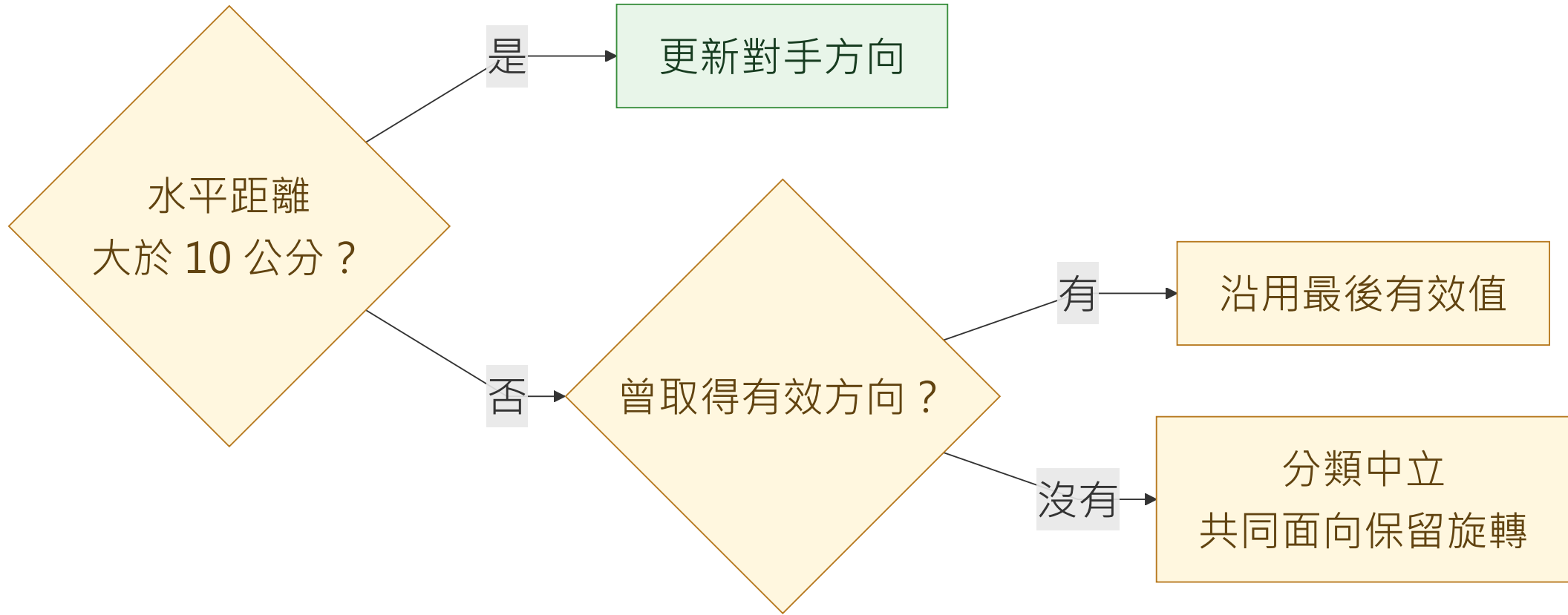
速度為 0 的分支：直接對正



符合面向條件時，首次面向或速度為 0，會直接套用目標角度。

程式定位 FightingCharacter.cpp : UpdateFacing() / bShouldSnap
 FightingCharacter.h : FacingInterpolationSpeed / bHasAppliedFacing

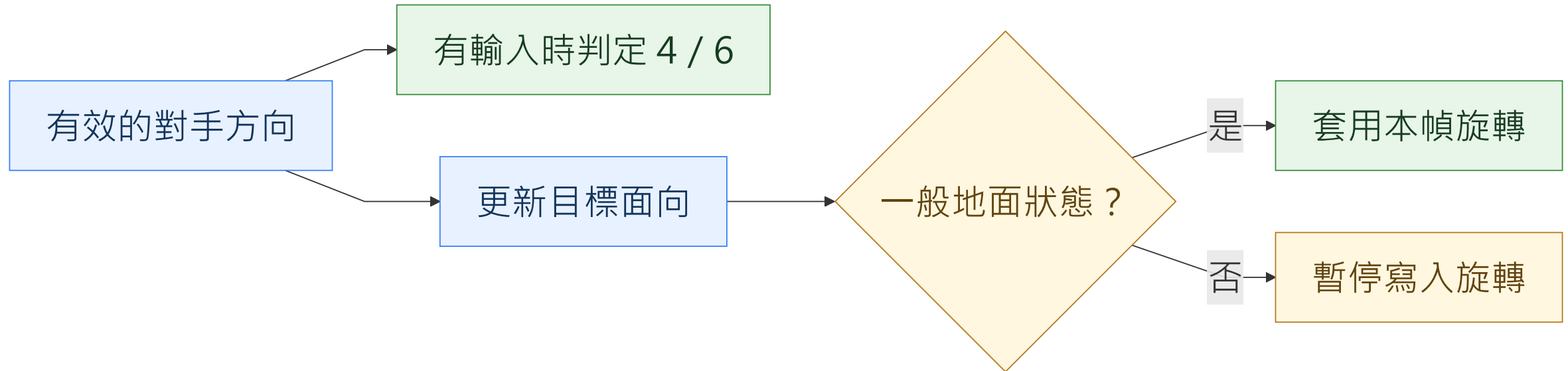
太靠近時，沿用最後有效方向



10 公分門檻保護方向資料；旋轉速度控制追向過程，兩者作用不同。

程式定位 FightingCharacter.cpp : UpdateHorizontalOpponentDirection() / MinimumOpponentDirectionDistanceCm
 FightingCharacter.h : LastValidOpponentDirection

方向持續更新，旋轉要看角色狀態

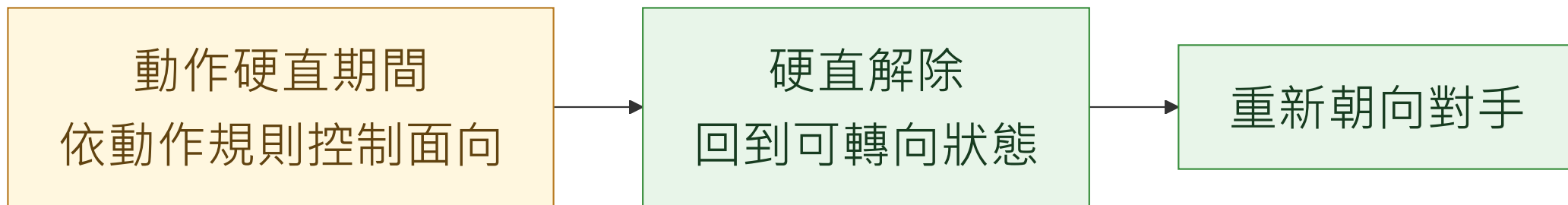


方向分類與目標角度可以更新；一般地面狀態才由共同面向寫入旋轉。

程式定位 FightingCharacter.cpp : Tick() / RefreshOpponentDirection() / UpdateFacing()
 FightingCharacter.cpp : IsInNormalGroundState()

格鬥遊戲自動轉身

待機、走路或防禦等狀態，可由系統自動面朝對手。



VF5 FS | 07:32

觀察對手位置改變後，角色何時重新轉正。

▶ 觀看影片

VF5 FS | 33:05

再看一段對戰，對照動作結束與面向更新。

▶ 觀看影片

教材對照：一般地面、無 Montage / Root Motion 時更新；轉速由插值控制。

硬直解除、回到可轉向狀態時，系統會重新面朝對手。

程式定位 FightingCharacter.cpp : IsInNormalGroundState() / UpdateFacing()
 FightingCharacter.h : DesiredFacingYaw / FacingInterpolationSpeed